

1. ภาพรวมโปรเจกต์

ระบบย่อลิงก์ (URL Link Shortener) ที่พัฒนาขึ้นเป็นโจทย์ทดสอบตำแหน่ง Full Stack Developer แบบครบวงจร ผู้ใช้กรอก URL ยาวเข้าไป ระบบจะคืนลิงก์สั้นที่มีแบรนด์กลับมา (เช่น gul.fy/HsQy5) พร้อมทั้งติดตามจำนวนครั้งที่ลิงก์ถูกเข้าถึง และเจ้าของลิงก์สามารถปิดใช้งาน (disable) หรือลบถาวรได้ นอกจากนี้ลิงก์สั้นหนึ่งลิงก์ยังสามารถ redirect ไปยังปลายทางที่แตกต่างกันได้ตามแพลตฟอร์มของผู้เข้าชม ไม่ว่าจะเป็น iOS, Android หรือแพลตฟอร์มอื่น ๆ

1.1 สิ่งที่ระบบทำได้

- สร้างลิงก์สั้นจาก URL ยาว พร้อมตั้งค่า custom alias เองได้ (ไม่บังคับ)
- สร้าง short code ได้ 2 วิธี — สุ่มอัตโนมัติ (auto-generated แบบ random base62) และแบบที่ผู้ใช้กำหนดเอง (custom alias)
- มีแดชบอร์ดแสดงลิงก์ทั้งหมด พร้อมจำนวนคลิก วันที่สร้าง และวันที่เข้าถึงล่าสุด
- ปิดใช้งานลิงก์ (disable) หรือลบถาวร (delete)
- ตั้งปลายทางแยกตามแพลตฟอร์ม — ลิงก์เดียวกันสามารถพาผู้ใช้ iOS, Android และผู้ใช้ทั่วไป ไปคนละปลายทางได้
- คัดลอกลิงก์ด้วยปุ่มเดียว (copy-to-clipboard) และดาวน์โหลด QR code ของลิงก์
- ใช้ in-memory storage — ไม่ต้องติดตั้งฐานข้อมูลใด ๆ เพื่อรันหรือตรวจโปรเจกต์

2. สิ่งที่ต้องเตรียมก่อนเริ่ม (Prerequisites)

เครื่องมือ	เวอร์ชัน	ใช้สำหรับ
.NET SDK	8.0 ขึ้นไป	ใช้ build และรัน backend (ASP.NET Core Web API) รวมถึงรัน unit test ฝั่ง backend
Node.js	20 ขึ้นไป	ใช้รัน Vite dev server ของ frontend (React) และรันชุดทดสอบฝั่ง frontend
npm	ติดมากับ Node	ติดตั้ง dependency ของ frontend
Git	เวอร์ชันล่าสุดทั่วไป	ใช้ clone repository

3. การดาวน์โหลด Code

Clone หรือแตกไฟล์ zip ของ repository แล้วเปิด terminal ที่ root ของโปรเจกต์ ทุกอย่างที่เป็นต่อการรันและทดสอบอยู่ใน repo เดียวครบถ้วน: <https://github.com/MoriMokata/url-link-shortener>

```
gulfy-link-shortener/  (repo root)
├── backend/           โซลูชัน .NET 8 (Api / Application / Domain / Infrastructure / Tests)
├── frontend/          แอป React + Vite + TypeScript
├── ai-logs/           บันทึก session AI และเอกสารดีไซน์ที่ export ไว้
└── README.md
```

4. Quick Start

4.1 เริ่มรัน Backend

เปิด terminal ที่ root ของ repo แล้วรันคำสั่ง:

```
cd backend
dotnet run --project Gulfy.Api --launch-profile http
```

คำสั่งนี้จะสตาร์ท Kestrel ที่ `http://localhost:5001` และเปิด Swagger UI ที่ `/swagger` ให้อัตโนมัติ ข้อมูลถูกเก็บแบบ in-memory เท่านั้น ดังนั้นข้อมูลจะรีเซ็ตทุกครั้งที่ process ถูกรีสตาร์ท

launch profile ตัวที่สองชื่อ `https` จะ bind ทั้ง `https://gul.fy:5001` และ `https://localhost:5001` ให้ตรงกับโดเมนที่แสดงในสกรีนช็อตของเอกสารนี้ แต่ต้องตั้งค่าไฟล์ `hosts` บนเครื่องก่อนหนึ่งครั้ง ดูรายละเอียดในหัวข้อ 9 (“การ map โดเมนบน local”)

4.2 เริ่มรัน Frontend

เปิด terminal ที่ root ของ repo แล้วรันคำสั่ง:

```
cd frontend
npm install
npm run dev
```

คำสั่งนี้จะเปิด Vite dev server ที่ `http://localhost:5173` ซึ่งจะ proxy ทุก request ที่ขึ้นต้นด้วย `/api/*` ไปที่ backend พอร์ต 5001 ให้อัตโนมัติ เปิด URL นี้ในเบราว์เซอร์เพื่อเริ่มใช้งานแอป

4.3 ตรวจสอบว่าใช้งานได้ถูกต้อง

เมื่อทั้งสอง process รันอยู่แล้ว:

- เปิด `https://gul.fy:5001/swagger/index.html` — ควรเห็น API contract ครบทั้ง 6 route ภายใต้ `/api/links` และ route redirect ที่ root level
- เปิด `http://localhost:5173` — แอป React ควรโหลดได้โดยไม่มี error ใน console และแสดงแดชบอร์ดว่างเปล่า (0 ลิงก์) ในการรันครั้งแรก
- ลองสร้างลิงก์จากหน้า UI (ดูหัวข้อ 6) แล้วตรวจสอบว่าลิงก์ปรากฏบนแดชบอร์ดและใช้งานได้จริง

5. การรันชุดทดสอบ (Test Suites)

5.1 Backend — xUnit

```
cd backend  
dotnet test
```

รันทุก test project ที่อยู่ในโซลูชัน ซึ่งในโปรเจกต์นี้คือ Gulfy.Tests ที่ครอบคลุมทั้งชั้น Domain, Application และ Infrastructure

5.2 Frontend — Vitest + React Testing Library

```
cd frontend  
npm test
```

รันเทสต์ระดับ component สำหรับ LinkForm (การ validate ฝั่ง client) และ LinkTable (การแสดงผลแถวลิงก์และสถานะ)

6. วิธีใช้งาน gul.fy

6.1 สร้างลิงก์สั้น

เปิดแท็บ “สร้างลิงก์” วาง URL ปลายทาง เลือกตั้ง custom alias เองได้ (ไม่บังคับ) และเปิดส่วน “ปลายทางตามแพลตฟอร์ม” เพื่อกำหนดปลายทางแยกสำหรับผู้ใช้ iOS และ Android ได้ตามต้องการ เมื่อกดยืนยันฟอร์ม ระบบจะเรียก POST /api/links และแสดงลิงก์สั้นที่ได้ พร้อมปุ่มคัดลอกและปุ่มสร้าง QR code

gul.fy สร้างลิงก์ แลบบอร์ด

สร้างลิงก์สั้นใหม่

แปลง URL ยาวให้เป็นลิงก์สั้น พร้อมตั้งค่า alias และปลายทางตามแพลตฟอร์มได้ตามต้องการ

URL ปลายทาง (บังคับ)

ต้องเป็น URL ที่ถูกต้อง (มี http:// หรือ https://)

Custom alias (ไม่บังคับ)

ใช้เฉพาะตัวอักษร a-z, A-Z, 0-9 และ - ... ยาวยาว 3-32 ตัวอักษร

ปลายทางตามแพลตฟอร์ม (ไม่บังคับ)

กำหนดปลายทางแยกสำหรับ iOS / Android — ถ้าไม่ตั้งค่า จะ redirect ไปที่ URL ปลายทางด้านบนเสมอ

+ สร้างลิงก์สั้น

✓ สร้างลิงก์สำเร็จ

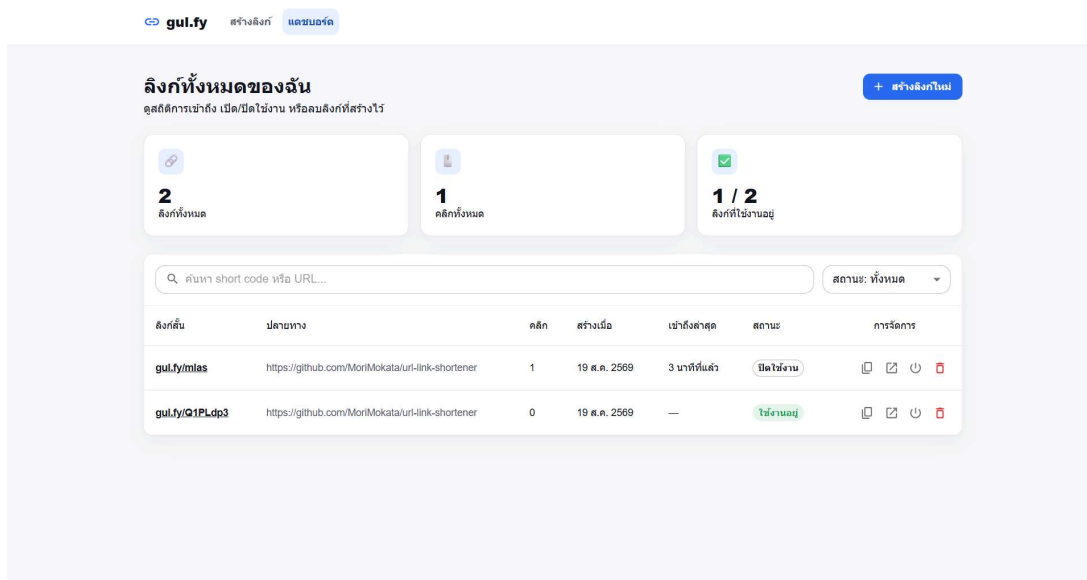
<https://gul.fy:5801/Q1Pldn3>

ต้นทาง: https://github.com/MoriMokata/url-link-shortener - iOS/Android ตั้งค่าแยกแล้ว

รูปที่ 1 — หน้าสร้างลิงก์: กรอก URL ปลายทาง, ตั้ง custom alias (ไม่บังคับ), ตั้งปลายทางแยกตามแพลตฟอร์ม (ไม่บังคับ) และผลลัพธ์หลังสร้างลิงก์สำเร็จ

6.2 แดชบอร์ด — ดูและจัดการลิงก์ทั้งหมด

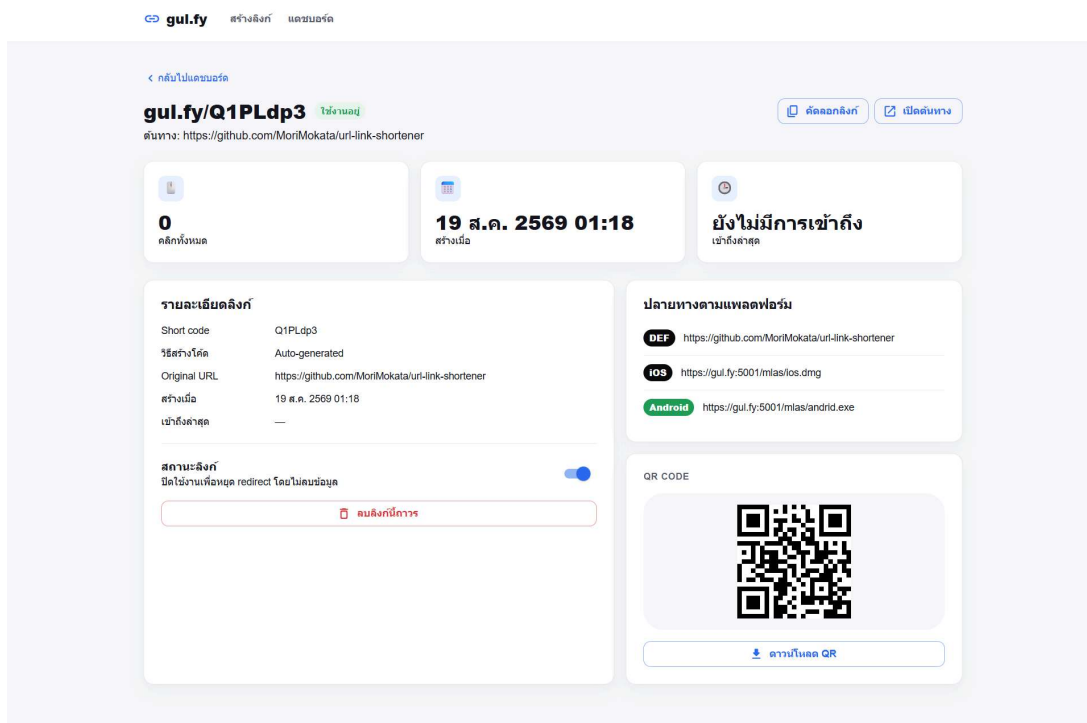
แท็บ “แดชบอร์ด” แสดงรายการลิงก์ทั้งหมดที่ยังไม่ถูกลบถาวร พร้อมช่องสรุปด้านบนแสดงจำนวนลิงก์ทั้งหมด จำนวนคลิกรวม และจำนวนลิงก์ที่ใช้งานอยู่ในปัจจุบัน แต่ละแถวแสดงลิงก์สั้น ปลายทาง จำนวนคลิก วันที่สร้าง และวันที่เข้าถึงล่าสุด พร้อมปุ่มจัดการด่วน — คัดลอกลิงก์ เปิดต้นทาง สลับสถานะเปิด/ปิดใช้งาน หรือลบลิงก์ ตารางจะสลับเป็นรูปแบบการ์ดโดยอัตโนมัติเมื่อหน้าจอแคบ (มือถือ)



รูปที่ 2 — แดชบอร์ด: ช่องสรุปข้อมูล, ช่องค้นหา/กรอง และตารางลิงก์พร้อมปุ่มจัดการในแต่ละแถว

6.3 รายละเอียดลิงก์ — สถิติ ปลายทางตามแพลตฟอร์ม และ QR code

คลิกที่ short code ของลิงก์จากแดชบอร์ดเพื่อเปิดหน้ารายละเอียด: จำนวนคลิกทั้งหมด วันที่สร้าง/เข้าถึงล่าสุด short code และวิธีที่ใช้สร้างโค้ด ปลายทางแยกตามแพลตฟอร์ม (ถ้ามีการตั้งค่าไว้) ปุ่มสลับเปิด/ปิดใช้งาน ปุ่มลบถาวร และ QR code พร้อมปุ่มดาวน์โหลดเป็นไฟล์ PNG



รูปที่ 3 — หน้ารายละเอียดลิงก์: สถิติ, ปลายทางตามแพลตฟอร์ม, ปุ่มสลับสถานะ และแผนผัง QR code

6.4 เกิดอะไรขึ้นเมื่อมีคนเปิดลิงก์สั้น

การเปิดลิงก์สั้นโดยตรงในเบราว์เซอร์ (เช่น <https://gul.fy:5001/Q1PLdp3>) จะไม่โหลดแอป React เลย แต่จะไปตรงที่ route redirect ระดับ root ของ backend แทนที่:

- ถ้าลิงก์ยังใช้งานอยู่ backend จะบันทึกการเข้าชม (เพิ่มจำนวนคลิก ปรับ “เข้าถึงล่าสุด”) ตัดสินใจปลายทางที่ถูกต้องตามแพลตฟอร์มของผู้เข้าชม แล้วส่ง 302 redirect ไปที่ปลายทางนั้น
- ถ้าลิงก์ถูกปิดใช้งานหรือถูกลบไปแล้ว หรือไม่พบโค้ดนั้นเลย backend จะคืนค่า 404 แทนที่จะ redirect

7. อ้างอิง API แบบย่อ

เอกสาร API แบบเต็มดูได้ที่ /swagger ขณะที่ backend กำลังรันอยู่ ข้อความ error ทุกกรณีใช้รูปแบบ RFC 7807 ProblemDetails ({ title, status, detail, instance })

Method	Route	คำอธิบาย	Error
POST	/api/links	สร้างลิงก์ — body { originalUrl, customAlias?, platformDestinations?: { ios?, android? } }	400, 409
GET	/api/links	ดึงรายการลิงก์ทั้งหมดที่ยังไม่ถูกลบ	—
GET	/api/links/{code}	ดึงรายละเอียดและสถิติของลิงก์เดียว	404
PATCH	/api/links/{code}/disable	ปิดใช้งาน — หยุด redirect แต่ข้อมูลยังอยู่	404

PATCH	/api/links/{code}/enable	เปิดใช้งานลิงก์ที่ถูกปิดกลับคืน	404
DELETE	/api/links/{code}	ลบถาวร	404
GET	/{code}	Redirect จริงที่ root level — คืน 302 ไปยังปลายทางพร้อมบันทึกคลิก	404

ตัวอย่าง — การสร้างลิงก์:

```
POST /api/links
Content-Type: application/json

{ "originalUrl": "https://www.google.co.th" }

→ 201 Created
{
  "shortCode": "HsQy5",
  "shortUrl": "https://gul.fy:5001/HsQy5",
  "originalUrl": "https://www.google.co.th",
  "source": "Auto",
  "isDisabled": false,
  "clickCount": 0,
  "createdAt": "2026-08-18T07:00:00Z",
  "lastAccessedAt": null,
  "platformDestinations": {}
}
```

8. อั่งอิงการตั้งค่า (Configuration)

การตั้งค่าทั้งหมดอยู่ที่ backend/Gulfy.Api/appsettings.json

การตั้งค่า	ค่าเริ่มต้น	วัตถุประสงค์
ShortUrl:BaseUrl	https://gul.fy:5001	prefix ที่ใช้สร้างค่า shortUrl ที่ API คืนกลับมา เปลี่ยนเป็น http://localhost:5001 ถ้าใช้ launch profile แบบ HTTP ธรรมดา
Cors:AllowedOrigins	["http://localhost:5173"]	origin ที่อนุญาตให้เรียก API จากเบราว์เซอร์ ได้ ปรับตามพอร์ต/โฮสต์ที่ frontend รันอยู่

9. การ Map โดเมน gul.fy บนเครื่อง Local

gul.fy ไม่ใช่โดเมนจริงที่จดทะเบียน จะใช้งานได้ก็ต่อเมื่อเครื่องถูกตั้งค่าไว้ล่วงหน้าเท่านั้น เป้าหมายเดียวคือให้ลิงก์
สั้นที่สร้างขึ้นแสดงผลและเปิดได้เหมือน gul.fy/HsQy5 จริง ๆ แทนที่จะเป็น localhost:5001/HsQy5 ขั้นตอนนี้ไม่
บังคับเลย ทุกพีเจอร์ทำงานเหมือนกันทุกประการผ่าน HTTP ธรรมดา

วิธีเปิดใช้งาน:

1. Map hostname ไปที่ loopback — แก้ไฟล์ hosts ด้วยสิทธิ์ Administrator/root แล้วเพิ่ม:

```
# C:\Windows\System32\drivers\etc\hosts บน Windows
# /etc/hosts บน macOS/Linux
127.0.0.1 gul.fy
```

2. Trust ใบรับรอง HTTPS dev บนเครื่อง (ทำครั้งเดียวต่อเครื่อง):

```
dotnet dev-certs https --trust
```

3. รัน backend ด้วย launch profile ชื่อ https (จะ bind ทั้ง https://gul.fy:5001 และ https://localhost:5001):

```
dotnet run --project Gulfy.Api --launch-profile https
```

ใบรับรอง dev มี CN เป็น localhost ไม่ใช่ gul.fy ดังนั้นเบราว์เซอร์จะยังคงแจ้งเตือนเรื่องใบรับรองเมื่อเปิดลิงก์ gul.fy — คลิก “Advanced → Proceed” เพื่อดำเนินการต่อได้เลย เป็นพฤติกรรมที่ถูกต้องตามที่คาดไว้ ไม่ใช่บั๊ก

10. การแก้ปัญหาเบื้องต้น (Troubleshooting)

อาการ	วิธีแก้
Frontend โหลดได้แต่ทุก request ล้มเหลว / ขึ้น CORS error ใน console	ตรวจสอบว่า backend รันอยู่จริง และ Cors:AllowedOrigins ใน appsettings.json มี origin ที่ frontend ถูกเสิร์ฟอยู่ตรงกัน (ค่าเริ่มต้นคือ http://localhost:5173)
เบราว์เซอร์แจ้งเตือนใบรับรองเมื่อเปิดลิงก์ gul.fy	เป็นเรื่องปกติเมื่อใช้ launch profile https — CN ของใบรับรอง dev คือ localhost ไม่ใช่ gul.fy คลิก “Advanced → Proceed” หรือข้ามการ map โดเมนแล้วใช้ profile http แทนก็ได้
พอร์ต 5001 หรือ 5173 ถูกใช้งานอยู่แล้ว	ปิดโปรเซสอื่นที่ใช้พอร์ตนั้น หรือเปลี่ยนพอร์ตใน launchSettings.json (backend) / vite.config.ts (frontend) แล้วปรับ ShortUrl:BaseUrl และ CORS origin ให้ตรงกัน
ลิงก์ทั้งหมดหายไปหลังรีสตาร์ท backend	เป็นพฤติกรรมปกติ — ข้อมูลเก็บแบบ in-memory เท่านั้น ไม่ถูก persist ข้ามการรีสตาร์ท process
dotnet test / npm test รัน restore package ไม่สำเร็จ	ตรวจสอบว่าเครื่องมี network ออกไปยัง nuget.org และ registry.npmjs.org ได้ ทั้งสองเป็น package registry มาตรฐาน

11. อ้างอิงโครงสร้างโปรเจกต์

11.1 Backend (backend/)

```
Gulfy.Domain      entity ShortLink, enum Platform/ShortCodeSource
Gulfy.Application DTOs, interface ของ service (ports), ShortLinkService, validation, exceptions
Gulfy.Infrastructure repository แบบ in-memory, ตัวสร้าง short code, ตัวตรวจจับแพลตฟอร์มจาก User-Agent
Gulfy.Api         Controllers, DI wiring, การจัดการ exception เป็น ProblemDetails, Swagger, CORS
Gulfy.Tests       unit test แบบ xUnit ครอบคลุมชั้น Domain/Application/Infrastructure
```

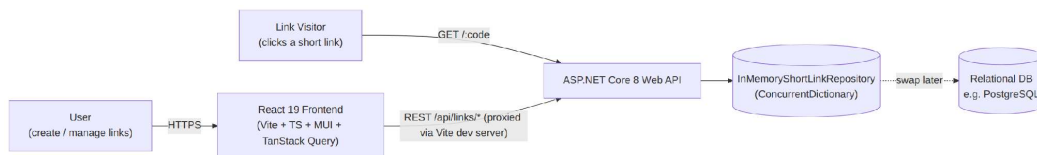
11.2 Frontend (frontend/src/)

```
api/      apiClient.ts (fetch wrapper), links.ts
types/    ShortLink, CreateShortLinkRequest
hooks/    useLinkMutations — disable/enable/delete พร้อม invalidate cache
components/ LinkForm, LinkTable, StatTile, CopyButton, QrCodeButton, QrCodePanel, AppHeader
pages/    CreateLinkPage, DashboardPage, LinkDetailPage
App.tsx   React Router: "/", "/dashboard", "/links/:code"
```

12. การออกแบบระบบ

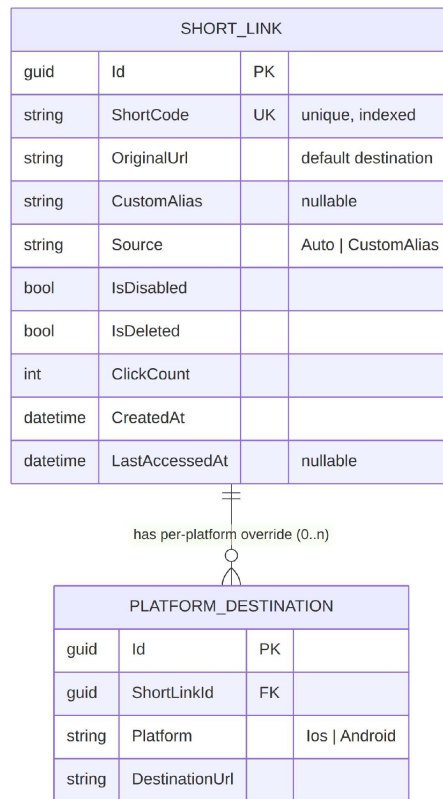
12.1 System context diagram

gul.fy เป็นระบบที่ประกอบด้วยแอป React แบบ single-page (UI สำหรับจัดการลิงก์) และ ASP.NET Core Web API (สร้าง/จัดการลิงก์ และ resolve redirect) สื่อสารกันผ่าน REST/JSON contract การเก็บข้อมูลปัจจุบันเป็นแบบ in-process memory แต่ออกแบบไว้เพื่อให้สามารถเปลี่ยนไปใช้ฐานข้อมูลจริงในภายหลังได้โดยไม่กระทบ business logic



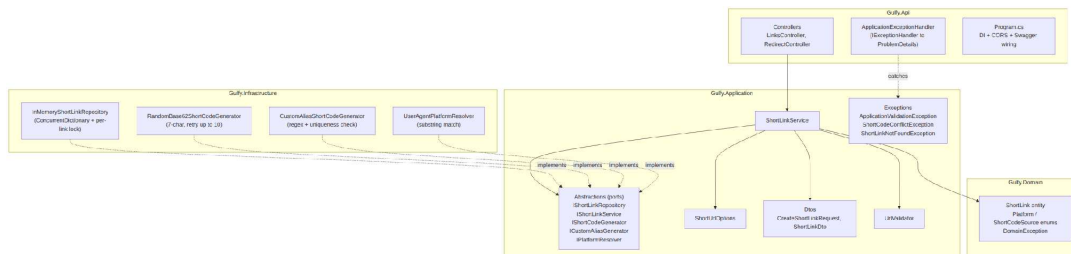
12.2 ER Diagram

ระบบมี aggregate เดียวคือ ShortLink ออกแบบให้เก็บ PlatformDestinations เป็น Dictionary<Platform,string> อยู่ภายใน entity เดียวกันในช่วง in-memory เพื่อความง่าย แต่รูปร่างของข้อมูลถูกออกแบบล่วงหน้าให้ map เป็นตารางแยก (normalized) ได้ทันทีเมื่อย้ายไปใช้ฐานข้อมูลเชิงสัมพันธ์ โดยไม่ต้องออกแบบ schema ใหม่



12.3 Backend Design

backend แบ่งเป็น 4 โปรเจกต์ (module) ตามชั้นสถาปัตยกรรม โดยมีทิศทางการพึ่งพา (dependency) จากบนลงล่างเท่านั้น: Api → Application → Domain และ Infrastructure → Application



12.3 Frontend Design

Frontend จัดโครงสร้างตามแนว “feature-oriented + layered”: api/ (การสื่อสารกับ backend), types/ (ชนิดข้อมูล), hooks/ (state/side-effect ที่ใช้ร่วมกันหลายหน้า), components/ (UI ที่ reuse ได้), pages/ (หน้าจอที่

ผูกกับ route) แต่ละชั้นพึ่งพาชั้นด้านล่างได้ทางเดียว (pages → components → hooks/api)

